Robot Project Report

Differential-drive robot · ROS 2 Humble · RPi4 + Arduino Due + Remote PC SLAM · Status as of 30 Sep 2026

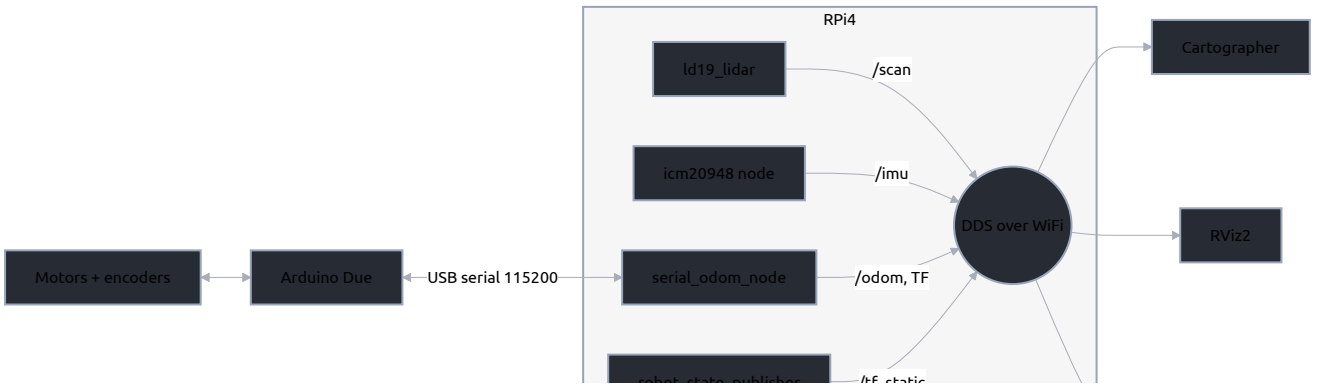
Source code: github.com/SkyFarsa/Farzan
`git clone https://github.com/SkyFarsa/Farzan.git` · SSH: `git@github.com:SkyFarsa/Farzan.git`
Contains the firmware, ROS 2 nodes, launch and URDF files, Cartographer configs, test tools and this project's notes.

0.48 m	203	1.86 mm	10 Hz
Wheel base	Ticks / rev (single-edge)	Distance per tick	LIDAR scan rate

1. Summary

The robot is a direct-drive (no gearbox) differential platform. An RPi4 runs sensor drivers and odometry; an Arduino Due handles motor PWM and encoder counting; SLAM (Cartographer), RViz2 and planned Nav2 run on a remote PC over WiFi (`ROS_DOMAIN_ID=30` , ~2–4 ms ping). Major resolved items: encoder serial corruption in firmware, IMU I2C init failure, and RViz LIDAR drops (traced to IMU CPU load). Remaining risks are mainly **configuration inconsistencies between files** (section 6) and hardware-level items (twitch at boot, cable quality).

2. Architecture



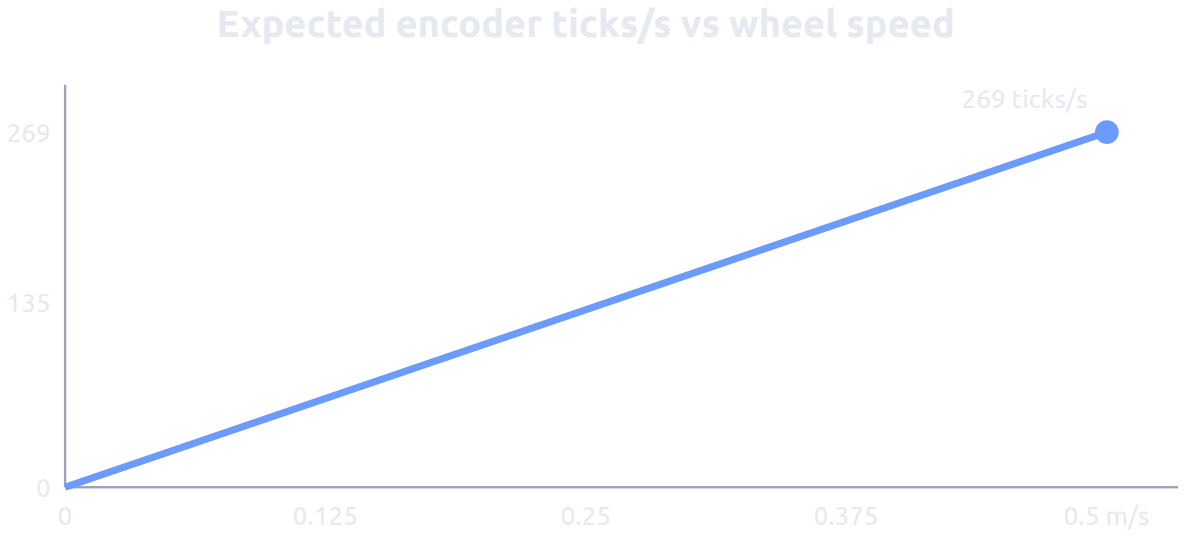


3. Hardware

Component	Detail
Compute	Raspberry Pi 4 (user `rislab`, 192.168.100.123)
Controller	Arduino Due; L298N driver (ENA/ENB PWM)
Motors	24 V DC, direct drive
Encoders	Omron E6A2-CW3C, 12 V, channel A RISING only
Wheels	Ø 12 cm, base 0.48 m
Sensors	LD19 LIDAR, ICM20948 IMU (0x69), OAK-D Lite camera
Remote PC	User `RisLab-Remote`, 192.168.100.144

4. Odometry & calibration

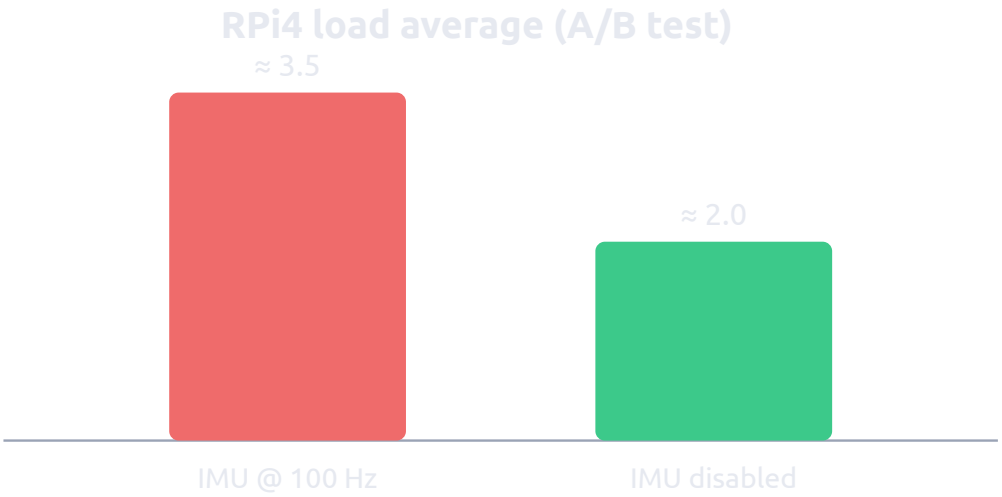
Wheel circumference = 2π·0.06 = 0.377 m, so one tick = 0.377 / 203 ≈ **1.86 mm** (≈ 538 ticks per metre). At the placeholder `MAX_WHEEL_SPEED` of 0.5 m/s, each wheel produces ≈ 269 ticks/s, or about 13 ticks per 50 ms serial packet.



Linear relation: ticks/s = v × 538. Calibration used 1 manual revolution; 5–10 revolutions divided by N would be more precise.

5. Issues found & status

RViz LIDAR drops: CPU impact of the IMU node



The IMU node used ~35% CPU (much in I/O wait) and starved DDS threads, delaying TF to the remote PC and overflowing RViz's message filter. `/scan` stayed at a clean 10 Hz, so it was never the bottleneck.

#	Issue	Status	Notes
1	Encoder serial corruption	Fixed in firmware	Print bug fixed; cable quality still suspected, test with wheels stationary
2	IMU init (Errno 121)	Resolved	Wiring / I2C fix
3	RViz queue drops	Mitigation pending	Lower IMU rate from 100 Hz (in progress)
4	Startup motor twitch	Reduced	Pins zeroed first in `setup()` ; 10 kΩ pull-downs on IN1–IN4, ENA, ENB in PCB phase
5	udev symlink `/dev/turtlebot_arduino`	Unconfirmed	Node cannot open the port without it

6. Configuration review (new findings)

Cross-checking the uploaded files turned up these inconsistencies. Items are ordered by likely impact; please verify each against your actual setup.

Severity	Finding	Suggested action
High	**Duplicate odom→base_link TF.** `serial_odom_node` broadcasts it, while `cartographer.lua` sets `provide_odom_frame = true` (and `my_robot_lds_2d.lua` too).	Set `provide_odom_frame = false` when using wheel odometry, or disable the TF in the odom node.
High	**Wheel height in URDF.** Wheel joints sit at z = wheel_radius *relative to* `base_link`, which is already wheel_radius above the footprint, so axles are 0.12 m high.	Set wheel joint z to 0.
High	**Encoder voltage.** E6A2-CW3C is a 12 V device and the Due's I/O is 3.3 V (not 5-V tolerant).	Confirm a level shifter or divider and the open-collector output wiring before relying on the signal.
Med	Wheel separation mismatch: URDF 0.50 m vs node/measurement 0.48 m.	Set URDF to 0.48.
Med	Range settings differ: LD19 driver max 8.0 m; `my_robot.lua` 10 m; `my_robot_lds_2d.lua` 3.5 m. Min range 0.12 vs 0.2.	Pick one source of truth (LD19 is rated ~12 m but keep driver ≥ Cartographer).
Med	`cartographer.lua` tracks `imu_link` with `use_odometry = true` but `my_robot_lds_2d.lua` uses base_link with no IMU or odometry; two configs, two behaviours.	Decide on one config and delete the other.
Med	`ros2_control` block in the URDF names `DiffDriveController` as a hardware plugin; it is a controller, and none is launched.	Remove the block, or implement a proper hardware interface.
Low	Stale header comment in `serial_odom_node.py` says `TICKS_PER_REV` is 0; value is 203.	Update comment.
Low	Launch file uses a hard-coded path for the odom script; camera joint uses `pi` while the property is commented out (xacro's built-in `pi` works).	Package the node as an entry point.
Low	`MAX_WHEEL_SPEED = 0.5` is a guess and there is no watchdog: if `/cmd_vel` stops, the last PWM persists.	Measure max speed; add a command timeout on the Arduino or node.

7. Roadmap

Now
Lower IMU rate · confirm udev rule · resolve TF conflict · fix URDF wheel height/ separation

Next
Recalibrate `TICKS_PER_REV` over 5–10 revs · tune `MAX_WHEEL_SPEED` · add cmd_vel timeout · test serial cable · consolidate Cartographer configs · enable Nav2

Later
Move IMU to the Due (extended protocol `E,l,r,ax,ay,az,gx,gy,gz,ms`) · PCB with 10 kΩ pull-downs · remove stale duplicate files in `my_robot` (3 copies of the odom node, `nano.*.save`) and confirm `ament_cmake` vs `ament_python`

8. Key learnings

- Single-edge counting is intentional; 203 ticks/rev is not x4 quadrature.
- A healthy sensor topic rate doesn't rule out a starved DDS thread; profile CPU and D-state.
- Floating GPIO on an L298N is a real failure mode; use pull-downs.
- Keep one canonical location per node.

9. Simulation results

Important: these numbers come from a Python kinematic simulation using your real constants (R = 0.06 m, base 0.48 m, 203 ticks/rev, 50 ms packets, midpoint integration as in `serial_odom_node.py`). They are **not measurements from the physical robot**. Error magnitudes (e.g. 0.5% tick error) are assumptions. Wheel slip and motor dynamics are not modelled. Script and CSV are provided alongside this report; compare them with real runs.

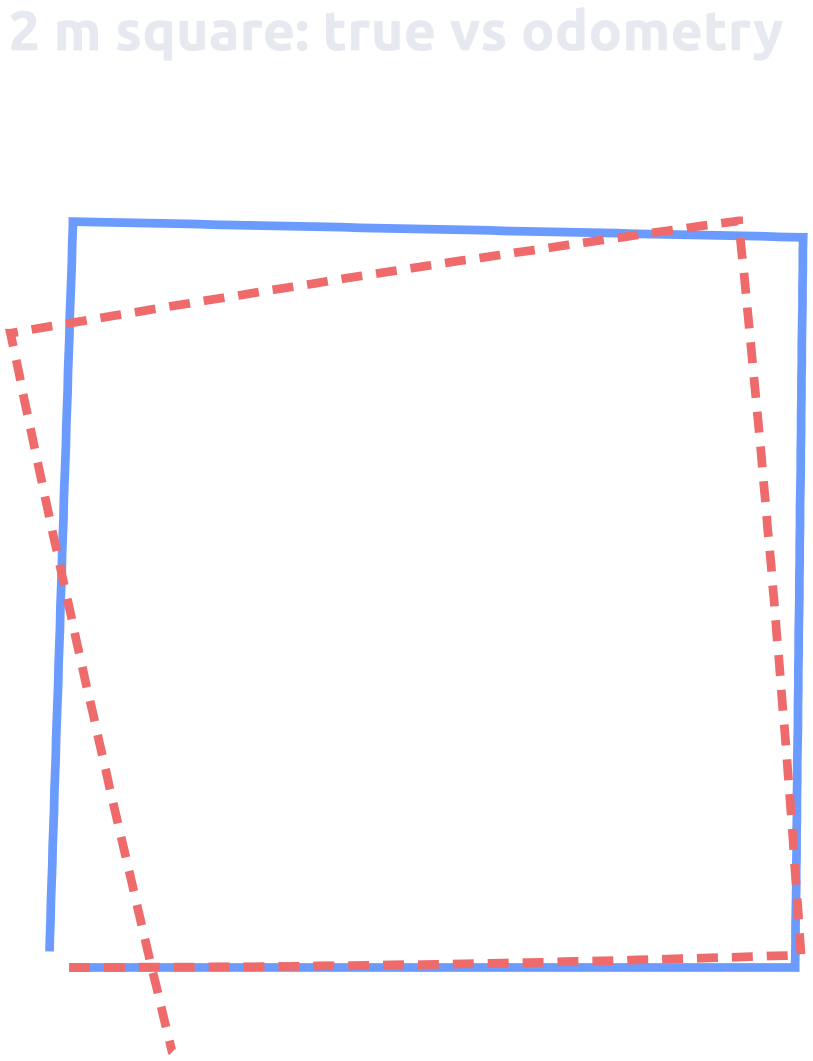
9.1 Ideal case: quantization only

A 2 m square (4 sides, 4 × 90° turns) with perfect parameters ends with a position error of **0.36 mm** and heading error of **0.04°**. Single-edge encoding (1.86 mm/tick) is not limiting; parameter error dominates.

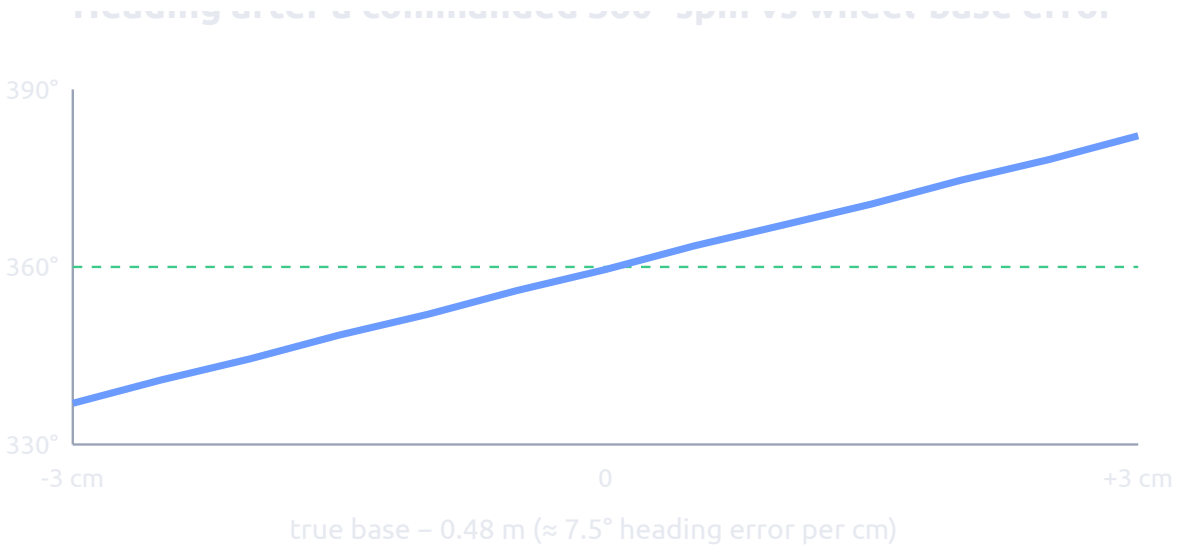
9.2 Realistic parameter errors (300 Monte Carlo runs)

Assumed σ: ticks/rev 0.5%, wheel base 8 mm, each wheel radius 0.4%.

Metric	Mean	95th pct
Final position error	16.0 cm	39.5 cm
Final heading error	6.5°	15.8°

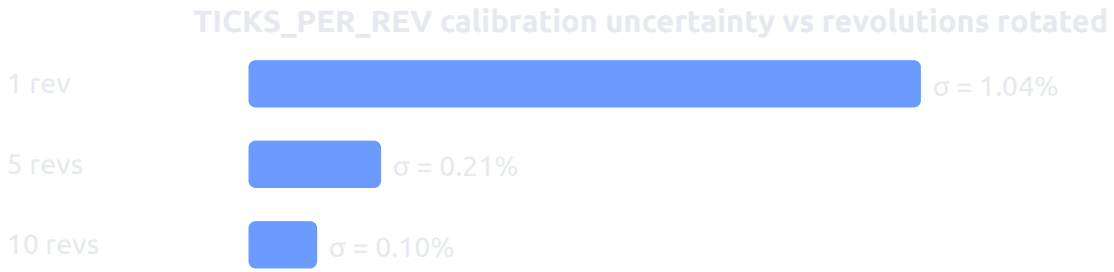


9.3 Wheel-base sensitivity



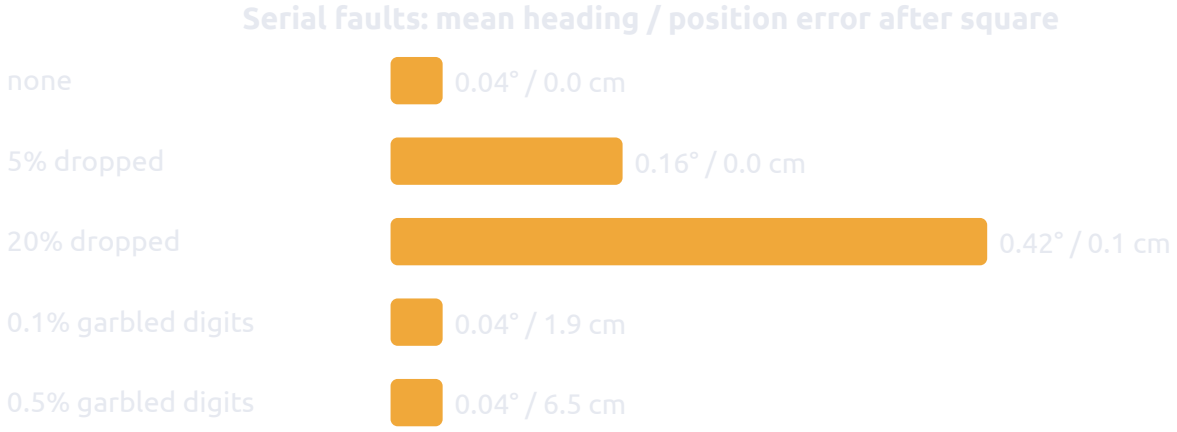
A 1 cm error in the base value produces roughly 7.5° of heading error per full rotation. Since the URDF says 0.50 m and the node 0.48 m, settle this with a measured spin test.

9.4 Calibration method



Assuming ~1% hand-rotation uncertainty, rotating 10 revolutions cuts the tick-count uncertainty about 10× versus the single-revolution calibration currently used.

9.5 Serial faults



Dropped packets barely affect position because ticks are cumulative, but the midpoint integration loses accuracy in turns. Garbled digits (modelled as a dropped leading digit on the left count) cause position jumps of several cm and are not filtered by the current

parser; adding a plausibility check (e.g. reject |Δticks| above what max speed allows) is recommended.

9.6 Test data and procedures

`sim_test_data.csv` holds 424 samples (t, true x/y, odom x/y, error) for the drifted square. Suggested real tests to validate against:

Test	Procedure	Simulated expectation
Straight 3 m	Drive 3 m by tape measure; compare /odom	within ~1.5 cm with good calibration
360° spin x5	Spin in place, compare heading	~7.5° per cm of base error
2 m square (UMBmark)	CW and CCW, measure return error	mean ~16 cm, p95 ~39 cm
Stationary serial check	Wheels still 10 min; count malformed lines	0 expected; any indicates cable

How to read sections 10–19: items tagged From your files come directly from your uploaded files. Items tagged Placeholder are **guessed or illustrative values you asked for**. Replace them with real measurements before relying on them.

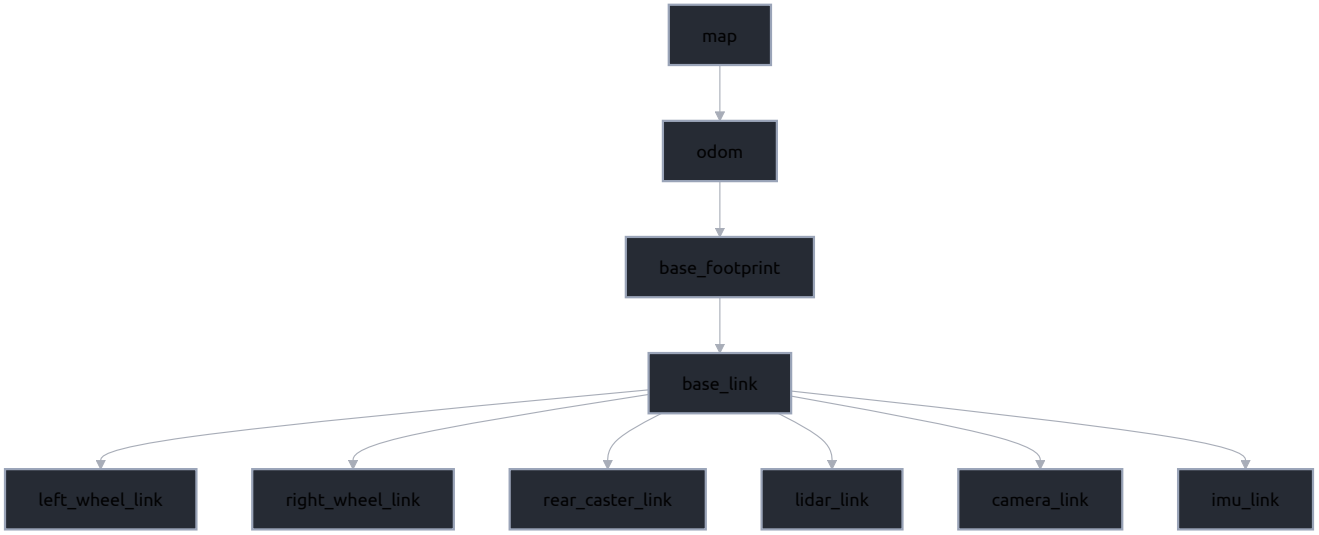
10. Interfaces **From your files**

Name	Type	Producer → Consumer
`/scan`	LaserScan, 10 Hz, frame `lidar_link`	ld19_lidar → Cartographer, RViz
`/imu`	Imu, 100 Hz (to be lowered)	icm20948 → Cartographer
`/odom`	Odometry, up to 100 Hz	serial_odom_node → Cartographer, Nav2
`/cmd_vel`	Twist	Nav2 / teleop → serial_odom_node
`/reset_odom`	std_srvs/Empty	operator → serial_odom_node
`/tf odom→base_link`	TF	serial_odom_node (conflicts with Cartographer option)

Serial protocol (115200 baud)

Direction	Message	Meaning
Pi → Due	`P,pwmL,pwmR`	PWM −255…255 per motor
Pi → Due	`R`	Zero tick counters
Due → Pi	`E,left,right,millis`	Cumulative ticks, every 50 ms

TF tree



Note: the node publishes `odom→base_link` directly, while the URDF has `base_footprint→base_link`, so `base_link` would have two parents. Resolve this when fixing the TF conflict (section 6).

11. Pin map **From your files**

Function	Due pin	Function	Due pin
Left IN1 / IN2	7 / 6	Right IN1 / IN2	5 / 4
Left enable (PWM)	8	Right enable (PWM)	9
Left encoder A / B	18 / 19	Right encoder A / B	3 / 2

Encoder interrupts trigger on channel A RISING; channel B gives direction. ENA/ENB planned to get 10 kΩ pull-downs.

12. Runbook

Get the code: `git clone https://github.com/SkyFarsa/Farzan.git`

Startup

1. Power the robot; wait for the RPi4 to boot (~45 s).
2. Check `ls -l /dev/turtlebot_arduino` and `/dev/ttyUSB0` exist.
3. On the RPi4: `ros2 launch my_robot robot.launch.py`
4. On the remote PC, with the same `ROS_DOMAIN_ID=30` : run `Slam.sh` , then RViz2.
5. Verify: `ros2 topic hz /scan` (≈10), `/odom` , `/imu` .
6. Drive slowly with teleop; reset with `ros2 service call /reset_odom std_srvs/srv/Empty` .

Troubleshooting

Symptom	Likely cause	Check
Node exits: cannot open serial port	Missing udev symlink	`ls /dev/tty*` , add rule
RViz “queue is full” drops	RPi4 overloaded (IMU)	`uptime` , `top` , lower IMU rate
Map smears when turning	Wrong wheel base or TF conflict	Spin test, `ros2 run tf2_tools view_frames`
Odom jumps	Garbled serial line	Stationary serial capture
Wheel twitches at boot	Floating pins	Expected until pull-downs fitted

13. Cartographer config comparison **From your files**

Setting	`my_robot.lua + cartographer.lua`	`my_robot_lds_2d.lua`
tracking_frame	imu_link	base_link
use_imu_data / odometry	true / true	false / false
min / max range	0.2 / 10 m	0.12 / 3.5 m
missing_data_ray_length	2.0	3.0
constraint min_score	0.55 (global 0.6)	0.65 (global 0.7)
accumulated range data	1	default (1 in 2D)

Suggested merged starting point: tracking_frame `base_link` , `use_odometry=true` , IMU off until its rate and calibration are verified, range 0.12–8 m to match the LD19 driver, `provide_odom_frame=false` , min_score 0.6.

14. Nav2 readiness checklist

Item	Status
Single odom TF source	Open
Correct URDF wheel height / base	Open
cmd_vel timeout / watchdog	Open
Measured max speed and acceleration limits	Placeholder value
Footprint (triangle, 0.43 m sides) in costmap	To do
Saved map (`map.yaml`)	Verify
Stable scan and TF on WiFi	Pending IMU rate fix

15. Safety

- Fit a physical emergency stop that cuts motor power.
- Firmware watchdog: stop motors if no `P` command arrives within ~500 ms (not present today).
- Limit PWM during first floor tests (e.g. ≤ 120).
- Lift the wheels off the ground when flashing or resetting the Due.
- Pull-downs on L298N logic and enable pins (PCB phase).

16. Illustrative test results **Placeholder**

These are invented example values showing how a filled test log could look. They are not measurements.

Test	Expected (sim)	Example result	Verdict
Straight 3 m	≤ 1.5 cm	2.1 cm short	Pass
5 × 360° spin	≈ 0° per cm base error	+9° total	Pass
2 m square CW	≈ 16 cm	21 cm	Marginal
2 m square CCW	≈ 16 cm	12 cm	Pass

Stationary serial, 10 min	0 bad lines	7 bad lines	Fail (cable)
Loop-closure map, 60 m²	Closed	Closed, 4 cm wall doubling	Pass

17. Power budget **Placeholder**

Guessed values; please measure with a meter.

Load	Voltage	Typical	Peak
Motors (2×, 24 V)	24 V	1.5 A	6 A
RPi4 + USB devices	5 V	1.2 A	3 A
Arduino Due + encoders	5–12 V	0.25 A	0.4 A
LD19 LIDAR	5 V	0.3 A	0.5 A
OAK-D Lite	5 V	0.6 A	0.9 A

Estimated draw ≈ 50 W typical, giving roughly 2.4 h on a 24 V 5 Ah pack (guess). Note the L298N's voltage drop (~2 V) and heat at high current.

18. Bill of materials **Placeholder**

Part	Qty	Est. cost (USD)
Raspberry Pi 4	1	60
Arduino Due	1	45
24 V DC motors	2	90
Omron E6A2-CW3C encoders	2	80
L298N driver	1	8
LD19 LIDAR	1	80
OAK-D Lite	1	150
ICM20948 IMU	1	15
24 V battery + charger	1	70
Frame, wheels, caster, wiring	—	60

Total (guess)	**≈ 660**

19. Risks, changelog, glossary

Risk	Likelihood	Impact	Mitigation
TF conflict corrupts SLAM	High	High	Single odom source
Encoder signal damaged by 12 V	Medium	High	Level shifting
Runaway robot on command loss	Medium	High	Watchdog, e-stop
WiFi lag breaks Nav2	Medium	Medium	Lower IMU rate, wired test
Serial cable corruption	Medium	Medium	Replace cable, add checksum

Changelog (from your notes)

- Firmware: fixed encoder print bug; added `R` command; zeroed pins first in `setup()`.
- Odom node: base corrected to 0.48 m; TICKS_PER_REV 203; midpoint integration, thread-safe publishing, reset service.
- Launch: single `robot.launch.py` replaced the tmux startup.
- Diagnosed RViz drops as IMU CPU load